



InfiniFi Protocol

Security Review



Disclaimer

Security Review

InfiniFi Protocol



The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

Table of Contents

Security Review

InfiniFi Protocol



Table of Contents

Summary	7
Scope	9
Methodology	9
Project Dashboard	13
Risk Section	16
Findings	18
3S--M01	18
3S--M02	20
3S--M03	21
3S--L01	23
3S--N01	25
3S--N02	27
3S--N03	29
3S--N04	31
3S--N05	32

Summary Security Review

InfiniFi Protocol



Summary

Three Sigma audited InfiniFi in a 0.8 person week engagement. The audit was conducted from 12/05/2026 to 13/05/2026.

Protocol Description

InfiniFi is a DeFi protocol that enables users to mint and redeem receipt tokens (iUSD, iETH) against collateral assets (USDC, ETH). The protocol features a sophisticated yield generation system through multiple farm integrations, a locking mechanism for enhanced rewards, and a governance system for farm allocation voting.

Scope Security Review

InfiniFi Protocol

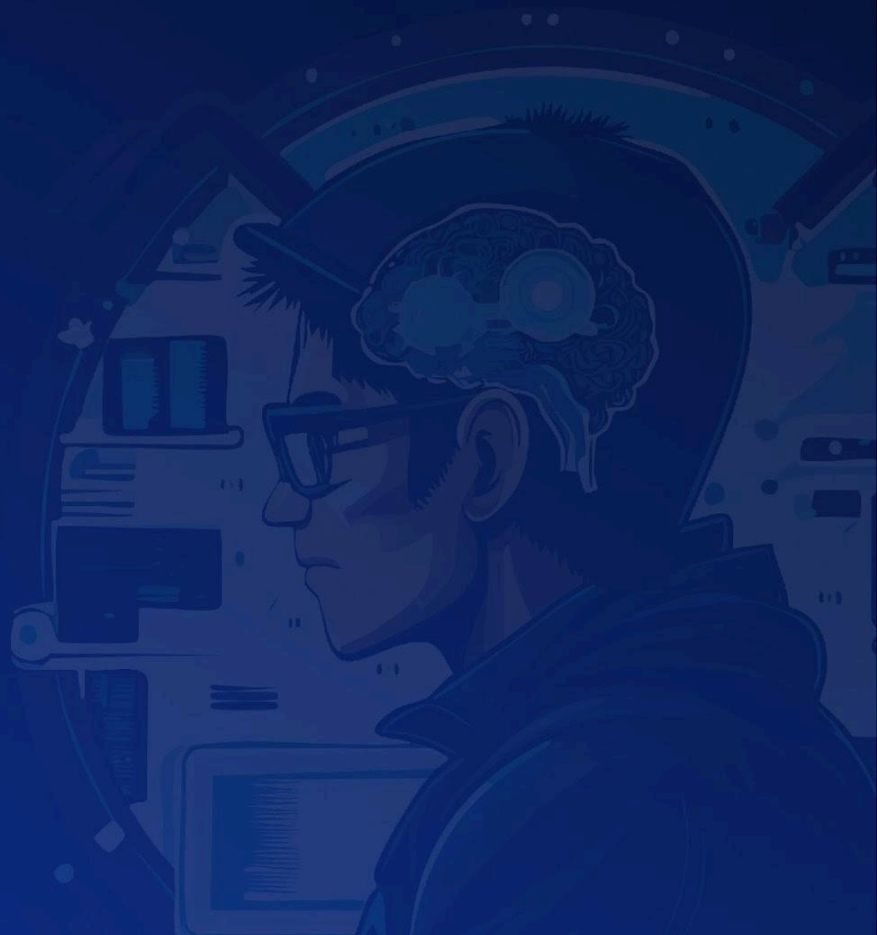


Scope

Filepath	nSLOC
PR349: src/integrations/farms/PrimeBrokerFarm.sol	143
PR359: src/tokens/GovernanceToken.sol	62
PR359: src/libraries/CoreRoles.sol	29
Total	234

Methodology Security Review

InfiniFi Protocol



To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

Taxonomy

In this audit, we classify findings based on ImmuneFi's [Vulnerability Severity Classification System \(v2.3\)](#) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

Impact / Likelihood	LOW	MEDIUM	HIGH
NONE	None		
LOW	Low		
MEDIUM	Low	Medium	Medium
HIGH	Medium	High	High
CRITICAL	High	Critical	Critical

Project Dashboard Security Review

InfiniFi Protocol



Project Dashboard

Application Summary

Name	InfiniFi
Repository	https://github.com/InfiniFi-Labs/infinifi-contracts
Commit	PR349, PR359
Language	Solidity
Platform	Ethereum

Engagement Summary

Timeline	12/05/2026 to 13/05/2026
Nº of Auditors	2
Review Time	0.8 person weeks

Vulnerability Summary

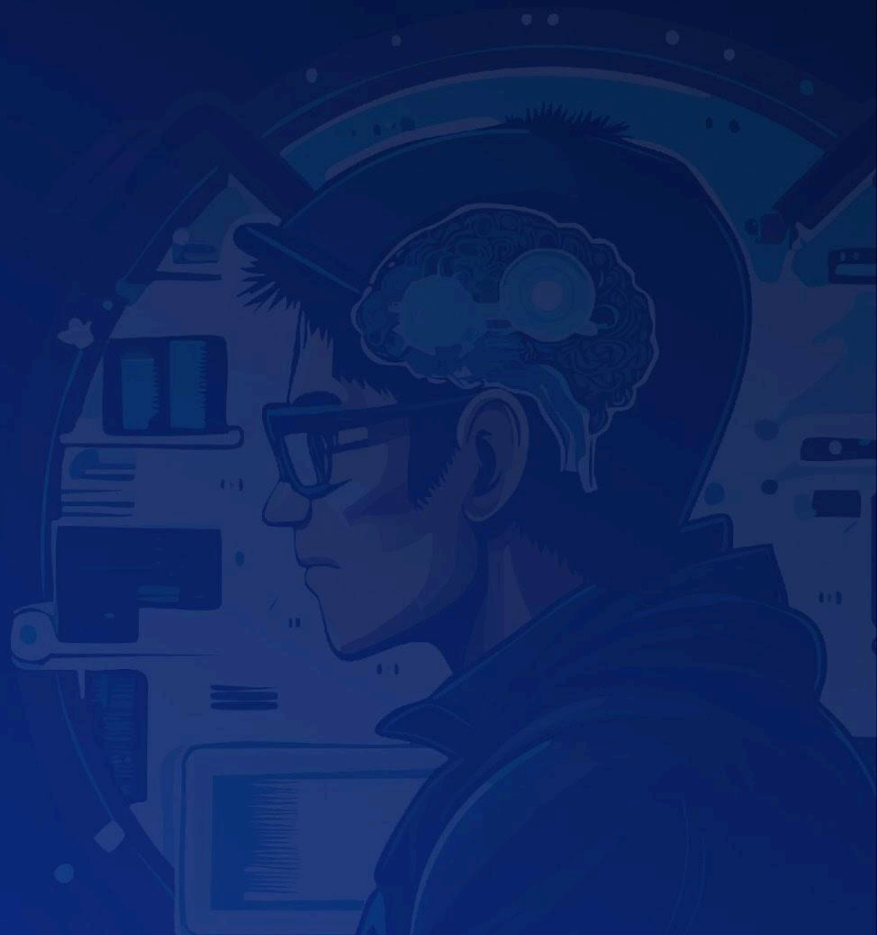
Issue Classification	Found	Addressed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	3	1	2
Low	1	1	0
None	5	4	1

Category Breakdown

Suggestion	5
Documentation	0
Bug	4
Optimization	0
Good Code Practices	0

Risk Section Security Review

InfiniFi Protocol



Risk Section

No risks were identified.

Findings

Security Review

InfiniFi Protocol



Findings

3S-M01

Re-checking the 3F whitelist inside **exitBid** can lead to DoS

Id	3S--M01
Classification	Medium
Impact	High
Likelihood	Low
Category	Bug
Status	Addressed in #75b4434 .

Description

PrimeBrokerFarm::exitBid is the redemption-side counterpart of **PrimeBrokerFarm::executeBid**. After 3F calls **setRepaid()** on a request, the backend keeper invokes **exitBid(_requestId, _destinationFarm)** to burn the PT/YT held by the farm and recover the underlying USDC (principal plus realised yield) back into the protocol.

Before performing any state changes, **exitBid** re-evaluates the same whitelist gate that guards entry: **require(IWhitelist(_WHITELIST).isWhitelisted(_requestId) == 1, RequestIdNotWhitelisted(_requestId))** (**PrimeBrokerFarm.sol:234**). The **IWhitelist** registry is owned by 3F, which is an external, trusted-but-mutable counterparty. Its status values are **Whitelisted (1)**, **NotWhitelisted**, **PausedWhitelisted**, and **PausedNotWhitelisted**. Only the literal value **1** passes the check.

This re-check creates a one-way DoS surface. Once **executeBid** has succeeded and **positions[_requestId]** is non-zero, the request is an active accounting entry on the farm side. If 3F subsequently transitions the request from **Whitelisted** to any other status (a deliberate pause, a routine retirement, a key rotation, or a unilateral policy change), every future call to **exitBid** for that request reverts on the whitelist line. The position can never be closed through the normal path: **positions[_requestId]** remains non-zero, **_deployedAssets** is never decremented, and the USDC owed to the farm stays locked inside the 3F request contract.

The whitelist's stated role in this contract is anti-spoofing at entry, ensuring **executeBid** only sends funds to a request address that 3F has acknowledged. Applying the same gate at exit conflates entry authorization with redemption rights, even though the position itself was already validated at entry time. The check protects against no value-flow attack on the exit

path (the burn target is fixed by **_requestId**, which is keyed in the **positions** mapping), so it adds liveness risk without adding security.

Recovery requires the **GOVERNOR** role to invoke **emergencyAction** to extract the trapped USDC from the 3F request, a privileged path that is not part of the normal lifecycle and that sidesteps the farm's accounting. Every open position at the moment of the flip is affected simultaneously.

Recommendation

Remove the whitelist re-check from **exitBid**. The non-zero **positions[_requestId]** invariant, combined with the entry-time whitelist gate in **executeBid**, is sufficient proof that the request was legitimately entered. Redemption should not depend on the continued whitelisted status of a counterparty that the farm is trying to disengage from.

```
function exitBid(address _requestId, address _destinationFarm)
    external
    nonReentrant
    whenNotPaused
    onlyCoreRole(CoreRoles.FARM_SWAP_CALLER)
{
-   require(!Whitelist(_WHITELIST).isWhitelisted(_requestId) == 1,
RequestIdNotWhitelisted(_requestId));
    uint256 deployedAmount = positions[_requestId];
    require(deployedAmount != 0, NoActivePosition(_requestId));
```

If anti-spoofing at exit is judged necessary for defence in depth, snapshot the whitelisted status into a per-**_requestId** flag at entry time and consult that snapshot in **exitBid**, so 3F-side status changes after entry do not alter farm-side liveness.

3S-M02

Vote delegation bypasses the pre-TGE transfer freeze in **GovernanceToken**

Id	3S--M02
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Acknowledged with note <i>"We do not plan to use the vote feature in the short term so that is fine (no other contracts will be present that will read the votes)."</i>

Description

GovernanceToken inherits from **ERC20Votes**, which itself inherits OpenZeppelin's **Votes** contract. **GovernanceToken::_update** gates transfers with the **transferEnabled** flag, so before the TGE only addresses listed in **canBypassTransferRestrictions** can move tokens. The delegation entry points **Votes::delegate** and **Votes::delegateBySig** sit outside that path and never call **_update**, so any holder can delegate voting power while the transfer freeze is active.

The freeze exists to keep both economic weight and governance weight under controlled distribution until the TGE. Before transfers are enabled, an early recipient can still hand voting power to any address. Token balances stay locked, but governance weight does not.

Recommendation

Override the internal **Votes::_delegate** hook so it applies the same bypass gate that **_update** already enforces. **Votes::_delegate** is the single point that both **delegate** and **delegateBySig** reach, so one override covers both entry points.

```
+ function _delegate(address _account, address _delegatee) internal override {
+   if (!transferEnabled) {
+     require(canBypassTransferRestrictions[_account], TransferDisabled());
+   }
+   super._delegate(_account, _delegatee);
+ }
```

+ }

3S-M03

Cap not enforced on YT yield component leads to silent cap bypass

Id	3S--M03
Classification	Medium
Impact	Medium
Likelihood	Medium
Category	Bug
Status	Acknowledged with note <i>"For this we rely on the allowance field."</i>

Description

PrimeBrokerFarm::executeBid deploys USDC into a 3F PrimeBroker auction by pulling liquidity from a source farm through **ManualRebalancer::singleMovement**, approving the request contract, and minting PT and YT tokens. The cap (a documented per-farm risk limit defined on the inherited **Farm** base) is intended to bound the total assets a farm can hold at any time, and is enforced by **PrimeBrokerFarm::deposit**, which is invoked from inside **singleMovement** and checks **assets() > cap** against **ERC20(assetToken).balanceOf(address(this)) + _deployedAssets**.

The cap check fires after USDC is pulled from the source farm but before **mint** returns the PT and YT tokens. At that point **_deployedAssets** has not yet been updated to reflect the YT component. After **mint**, line 208 increments **_deployedAssets** by **ptReceived + ytReceived**, and no second cap check is performed. Because **ytReceived** is a positive quantity on any yield-positive bid (and **ytReceived > 0** is required by the check at line 202), every successful **executeBid** call raises **assets()** above the cap by exactly **ytReceived**.

The effect compounds across bids. A single **executeBid** with **_amount = cap** and any positive yield produces **assets() > cap** immediately, and subsequent bids increase the overshoot by $\Sigma \text{ytReceived}_i$. The cap remains nominal rather than binding, which weakens the protocol's documented blast-radius bound and contributes to escalation chains where the cap is the last line of defense against malicious counterparty behavior (such as a whitelisted-but-malicious request contract underpaying at exit).

Recommendation

Re-check the cap after the post-mint accounting update inside **executeBid**. The simplest fix is to add a check immediately after the **_deployedAssets += ptReceived + ytReceived;** statement at line 208.

```
positions[_requestId] += ptReceived + ytReceived;  
_deployedAssets += ptReceived + ytReceived;  
+ if (assets() > cap) revert CapExceeded(assets(), cap);  
+  
  if (_smoothingDuration != 0) {
```

3S-L01

Missing **_amount > 0** guard in **executeBid** allows magic-value sweep of source farm into untracked residue

Id	3S--L01
Classification	Low
Impact	Medium
Likelihood	Low
Category	Bug
Status	Addressed in #5506658 .

Description

PrimeBrokerFarm::executeBid is the entry-side hook used by the backend keeper to deploy USDC into an open 3F auction. The keeper passes a source farm, a request contract, an amount of USDC, and a minimum yield, and the function pulls the USDC out of the source farm via **ManualRebalancer::singleMovement**, approves it to the request, and calls **IPrimeBroker::mint** to receive PT and YT tokens. The PT amount is reconciled against the requested **_amount** and the position is recorded as **ptReceived + ytReceived**.

executeBid never validates that **_amount** is non-zero. It is forwarded as-is into **ManualRebalancer::singleMovement**, whose contract documents two magic values in the comment at **ManualRebalancer.sol:94**: an **_amount** of zero is interpreted as "move all of **liquidity()** from the source farm", and an **_amount** of **type(uint256).max** is interpreted as "move all of **assets()**". The branch at **ManualRebalancer.sol:103-107** implements that overload. When a keeper passes **_amount = 0** to **executeBid**, **singleMovement** reads **IFarm(_from).liquidity()** and pulls the entire source-farm liquidity into the PB farm.

The downstream **mint** call only consumes whatever USDC was authorized by 3F. If 3F has authorized a pathological (**ptMintAuth = 0**, **ytMintAuth > 0**) minting (a configuration the contract does not forbid), **IPrimeBroker::mint** pulls zero USDC and returns (**ptReceived = 0**, **ytReceived = ytMintAuth**). The **require(ptReceived == _amount, ...)** check at **PrimeBrokerFarm.sol:198** passes because both sides equal zero, the **ytReceived > 0** check passes, and the call succeeds. **positions[_requestId]** is recorded as **ytReceived** only, and **_deployedAssets** is incremented by the same amount.

The swept USDC remains on **PrimeBrokerFarm** as raw token balance that no position accounts for. Recovery requires the **MANUAL_REBALANCER** role to manually move the funds back.

Recommendation

Add a single-line guard at the top of **executeBid** that rejects the zero sentinel before it reaches **singleMovement**. The same change should also reject the **type(uint256).max** sentinel, which is harvested by **singleMovement** as "move all of **assets()**" and would create an analogous misclassification on any farm where **assets()** exceeds **liquidity()**. Bounding the upper end at **type(uint128).max** rather than **type(uint256).max - 1** also closes the silent truncation on the **uint128(_amount)** cast at **PrimeBrokerFarm.sol:193**, matching the input domain of the downstream **IPrimeBroker::mint**.

```

    ) external nonReentrant whenNotPaused
    onlyCoreRole(CoreRoles.FARM_SWAP_CALLER) {
        require(!Whitelist(_WHITELIST).isWhitelisted(_requestId) == 1,
RequestIdNotWhitelisted(_requestId));
+       require(_amount > 0 && _amount <= type(uint128).max,
InvalidAmount(_amount));
        require(positions[_requestId] == 0, PositionAlreadyActive(_requestId));

```


3S-N01

Stale OFT references remain in **GovernanceToken** NatSpec and **mint** mainnet gate

Id	3S--N01
Classification	None
Category	Suggestion
Status	Addressed in #c463d28 .

Description

GovernanceToken no longer inherits from **OFTCore** after PR #359 removed the LayerZero integration, yet two artifacts from the OFT era survive in the source:

1. The contract-level NatSpec still tells the reader to check **this.endpoint().delegates(this)** and treats the deployer as "the OApp delegate". Neither the LayerZero endpoint nor the OApp delegate role exist on the contract anymore.
2. **GovernanceToken::mint** keeps the **require(block.chainid == 1)** guard along with the comment "only allow minting on L1, the supply shall be created on L1 and then bridged to L2s through the OFT logic." PR #359 deleted the OFT bridge path, so the comment now contradicts the code, and the guard silently pins **mint** to Ethereum mainnet without any documented rationale.

The chain ID guard itself comes from the spearbit-21 fix that intentionally blocked L2 minting under the OFT design. The codebase does not record whether that intent still applies post-OFT, so a future deployer reading the file cannot tell if the restriction is load-bearing or dead leftover.

Recommendation

Drop the LayerZero language from the NatSpec, and rewrite the comment inside **GovernanceToken::mint** so it documents the current mainnet restriction in present-tense terms. If multi-chain INF1 deployment is acceptable, remove the **require(block.chainid == 1)** check instead.

```
/// @notice InfiniFi Governance Token.
/// This contract is meant to be used for INF1 & wINF1 tokens.
-/// @dev !\ This contract is initially owned by the deployer, and the deployer is set as
the OApp delegate,
```

-/// to enable easier initial configuration. Make sure to transfer the privileges before enabling it in the protocol.

-/// This can be checked by reading this.owner() and this.endpoint().delegates(this).

+/// @dev !\ This contract is initially owned by the deployer to enable easier initial configuration.

+/// Transfer ownership before enabling it in the protocol. This can be checked by reading this.owner().

contract GovernanceToken is CoreControlled, ERC20Burnable, ERC20Votes, Ownable {

**function mint(address _to, uint256 _amount) external
onlyCoreRole(_MINTER_ROLE) {**

- // only allow minting on L1, the supply shall be created on L1 and then

- // bridged to L2s through the OFT logic.

+ // restrict the entire INFI supply to Ethereum mainnet; deployments on other chains must not mint.

require(block.chainid == 1);

_mint(_to, _amount);

}

3S-N02

renounceOwnership call before **enableTransfers** leads to permanent freeze of all governance tokens

Id	3S--N02
Classification	None
Category	Suggestion
Status	Addressed in #bc7e2e6 .

Description

GovernanceToken is the shared implementation behind INFI and the four wINFI bucket share tokens. It is launched non-transferable (**transferEnabled == false**) so that the protocol can stage initial distribution, and the owner is expected to flip the switch via **GovernanceToken::enableTransfers** once the TGE is complete. Until that happens, every transfer reverts with **TransferDisabled** unless the sender appears in **canBypassTransferRestrictions**, which is populated only through the **onlyOwner** setter **GovernanceToken::enableTransfersForAddress**.

The contract inherits OpenZeppelin's single-step **Ownable**, which exposes a public **renounceOwnership()** that sets **_owner** to **address(0)**. Because both **GovernanceToken::enableTransfers** and **GovernanceToken::enableTransfersForAddress** are guarded by **onlyOwner**, calling **renounceOwnership()** while **transferEnabled == false** permanently removes the only key that can lift the transfer restriction or whitelist additional bypass addresses. The **Core** governance plane (which holds the minter and burner roles consumed by **onlyCoreRole**) is orthogonal to **Ownable** and cannot reassign ownership, so there is no on-chain recovery primitive.

A single accidental or coerced **renounceOwnership()** call before the TGE switch is flipped therefore freezes all five token instances (INFI and the four wINFI buckets) for every non-whitelisted holder, with no way to recover short of redeploying the entire token set and migrating balances.

Recommendation

Override **renounceOwnership()** on **GovernanceToken** so it cannot be invoked while transfers are still disabled, closing the pre-launch brick path while still allowing the owner to renounce after the distribution switch has been flipped:

```
+ function renounceOwnership() public override onlyOwner {  
+   require(transferEnabled, TransfersNotEnabled());  
+   super.renounceOwnership();  
+ }
```

3S-N03

Hardcoded **_WHITELIST** constant without admin override

Id	3S--N03
Classification	None
Category	Suggestion
Status	Addressed in #75b4434 .

Description

PrimeBrokerFarm integrates with 3F's PrimeBroker by gating every entry and exit through the 3F Request Whitelist, a registry contract owned by 3F that maps a request-id address to a **WhitelistStatus** enum. The farm queries this registry at **PrimeBrokerFarm.sol:170** inside **PrimeBrokerFarm::executeBid** and at **PrimeBrokerFarm.sol:234** inside **PrimeBrokerFarm::exitBid** via **IWhitelist(_WHITELIST).isWhitelisted(_requestId) == 1**, ensuring that funds are only deployed against requests 3F has acknowledged.

The registry reference itself is declared as **address public constant _WHITELIST = 0x3FcD87948cBF46605D6ded0ed56d3daCcd9daf9e** at **PrimeBrokerFarm.sol:77**. Because the value is **constant**, it is baked into the deployed bytecode at compile time. There is no setter, no **immutable** constructor parameter, no per-instance configuration, and no admin path (through **PROTOCOL_PARAMETERS**, **GOVERNOR**, or any other role) to change it post-deployment. Every other piece of farm configuration that touches the 3F integration (cap, allowance, smoothing duration, role assignments, pause state, and the broker request address itself) is governable; the address of the registry that gates every entry and every exit is not.

3F retains the ability to redeploy, version, or selfdestruct-and-recreate their whitelist contract behind their own upgrade machinery. A migration to a new address, a CREATE2 redeploy that the farm cannot follow, or a chain-level move all leave the hardcoded constant pointing at a stale or empty contract. From the farm's perspective the failure is indistinguishable from a global de-whitelisting: entry is blocked, and every open position is simultaneously locked out of the normal exit path because the same require at line 234 reverts. Recovery for each individual stuck position then requires **GOVERNOR** to invoke **emergencyAction** against the request contract directly, a privileged escape hatch that bypasses farm-side accounting in **positions** and **_deployedAssets**.

Recommendation

Replace the **constant** declaration with a configuration value that can be set after deployment. The simplest fix mirrors the pattern already used by

PrimeBrokerFarm::setAllowance: convert **_WHITELIST** to a storage slot updatable via a **PROTOCOL_PARAMETERS**-gated setter. If post-deployment mutability is undesired for governance reasons, the next-best option is to make the address **immutable** and assign it in the constructor so a single codebase can be deployed across chains without recompilation.

```
- address public constant _WHITELIST =
0x3FcD87948cBF46605D6ded0ed56d3daCcd9daf9e;
+ address public whitelist;
+
+ function setWhitelist(address _whitelist) external
onlyCoreRole(CoreRoles.PROTOCOL_PARAMETERS) {
+     whitelist = _whitelist;
+ }
```

3S-N04

Test fixture bucket parameters diverge from production TGE configuration, leaving production lock buckets and multipliers untested

Id	3S--N04
Classification	None
Category	Suggestion
Status	Acknowledged with note “We don't think the absolute value of the number of weeks & the multipliers (all within the same order of magnitude) are needed to be tested as-is in the unit test suite.”

Description

The Foundry test fixture in **Fixture.t.sol** provisions the INFI locking module that will hold time-locked governance tokens (**IINFI**) after the Token Generation Event. Each bucket pairs an **unwindingEpochs** value (lock duration in weeks) with a reward multiplier and a dedicated **IINFI-Nw** ERC-20, configured through **InfliLockingController::enableBucket**. The same configuration is replayed in production by **Proposal_TGE_1** when it pushes entries into **INFI_BUCKET_EPOCHS** and **INFI_BUCKET_MULTIPLIERS** and then calls **enableBucket** for each pair.

The two configurations do not match. **Fixture.t.sol:328-346** deploys buckets at {4, 8, 12, 16} weeks with multipliers {1.0x, 1.1x, 1.2x, 1.3x} (computed as $1e18 + 0.1e18 * (i - 1)$). **Proposal_TGE_1.sol:39-46** deploys buckets at {4, 8, 13, 26} weeks with multipliers {1.0x, 1.25x, 1.5x, 2.0x}. Two of the four production durations (13 and 26 weeks) are never exercised by the test suite, and all three non-trivial multipliers differ.

The practical consequence is that any test referencing **shareToken(12)** or **shareToken(16)**, or asserting reward math using the $0.1e18 * (i - 1)$ multiplier curve, exercises buckets that will not exist on mainnet. Arithmetic that is sensitive to the steeper production multipliers (for example reward accrual, weighted voting power, or unwinding accounting at 2.0x) is not covered by the existing tests, and regressions introduced into bucket-dependent code paths can pass CI while still being broken under the deployed parameters. This is a test-quality and process issue rather than an exploitable on-chain defect, but it reduces the assurance the suite provides over the actual TGE deployment.

Recommendation

Define the bucket configuration in a single shared source (for example by importing the **INFI_BUCKET_EPOCHS** and **INFI_BUCKET_MULTIPLIERS** arrays from **Proposal_TGE_1** into **Fixture.t.sol**, or by moving both into a common library) so the fixture and the deployment proposal cannot drift.

3S--N05

GovernanceToken::enableTransfersForAddress does not emit an event

Id	3S--N05
Classification	None
Category	Suggestion
Status	Addressed in #293ea9c .

Description

GovernanceToken::enableTransfersForAddress lets the owner grant or revoke an address's privilege to move tokens during the pre-TGE transfer freeze by writing to **canBypassTransferRestrictions**. The function mutates state silently and emits no event, while its sibling **GovernanceToken::enableTransfers** does emit **TransfersEnabled** when the global flag flips.

Indexers and monitoring tools that consume event logs cannot tell who held the bypass privilege at any past block without replaying contract storage. The bypass list controls who can move INFI before the public TGE.

Recommendation

Declare a new event and emit it from **GovernanceToken::enableTransfersForAddress** on every write, including the previous value so consumers can distinguish grants from revocations.

```
+ /// @notice emitted when a transfer-restriction bypass flag is updated for an
+ address
+ event TransferRestrictionBypassUpdated(address indexed account, bool
+ oldValue, bool newValue);
+
+ /// @notice enable transfers for a specific address before transfers are enabled for
```



```
all
    function enableTransfersForAddress(address _address, bool _value) external
onlyOwner {
+     bool _previous = canBypassTransferRestrictions[_address];
        canBypassTransferRestrictions[_address] = _value;
+     emit TransferRestrictionBypassUpdated(_address, _previous, _value);
    }
```